YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

LOW LEVEL DESIGN

ON

CLOUD COST INTELLIGENCE FOR E-COMMERCE

SUBMITTED BY

MUHAMMAD NIHAL

REG NO :

23BCAICD076

LH-06

GUIDED BY

MISS SINDHU

SUBMISSION DATE

17-04-2026

Table of Contents

1 Introduction
- 1.1 Scope of the document
- 1.2 Intended Audience
- 1.3 System overview

2 System Design
- 2.1 Application Design
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
- 2.5 Key Design Considerations
- 2.6 API Catalogue

3 Data Design
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policies
- 3.4 Data Migration

4 Interfaces

5 State and Session Management

6 Caching

7 Non-Functional Requirements
- 7.1 Security Aspects
- 7.2 Performance Aspects

8 References

1. Introduction

1.1 Scope of the Document

This document provides a detailed technical description of the implementation of the Cloud Cost Intelligence System. It explains how data is processed, how features are derived, how machine learning is applied, and how results are generated and visualized at a code level.

1.2 Intended Audience

This section is intended for developers, technical reviewers, and evaluators who require a deeper understanding of the internal logic, algorithms, and implementation details of the system.

1.3 System Overview

At the implementation level, the system operates as a data processing pipeline where raw cloud usage data is transformed into insights using:

- Mathematical cost calculations
- Statistical threshold-based rules
- Machine learning model (Isolation Forest)
- Rule-based recommendation engine

2. System Design

2.1 Application Design

The application is implemented as a **single Streamlit script (app.py)** with clearly defined logical sections:

- UI configuration (st.set_page_config)
- Data loading
- Feature engineering
- Filtering
- Waste detection
- Anomaly detection
- Recommendation generation
- Visualization

This linear structure ensures readability and ease of debugging.

2.2 Process Flow

Step 1: Data Loading

- Dataset is loaded using:

```
df = pd.read_csv("outputs/reports/ecommerce_cloud_cost_dataset.csv")
```

Step 2: Feature Engineering (Cost Calculation)
New columns are created using fixed pricing formulas:

- Compute Cost

```
compute_cost = compute_hours * 0.05
```

- Storage Cost

```
storage_cost = storage_usage_gb * 0.01
```

- Transfer Cost

```
transfer_cost = data_transfer_gb * 0.02
```

- API Cost

```
api_cost = api_requests * 0.0001
```

- Total Cost

```
total_cost = compute_cost + storage_cost + transfer_cost + api_cost
```

Step 3: Service Mapping
Service types are mapped to readable cloud services:

- Compute → EC2 / VM
- Storage → S3
- Database → RDS
- ML → SageMaker

Step 4: Filtering Logic

- User selects departments using Streamlit sidebar
- Data is filtered using:

```
df = df[df['department'].isin(selected_dept)]
```

Step 5: Waste Detection Logic

Thresholds are computed using quantiles:

- Low CPU threshold → 25th percentile
- High Cost threshold → 75th percentile
- Low Users threshold → 25th percentile
- High Storage threshold → 75th percentile

Waste conditions:

```
is_waste = TRUE if:
1. cpu_utilization < low_cpu AND compute_hours > 5
OR
2. total_cost > high_cost AND active_users < low_user
OR
3. storage_usage_gb > high_storage AND active_users < low_user
```

Waste cost calculation:

```
waste_cost = total_cost * 0.5 (only for waste rows)
```

Step 6: Anomaly Detection (Machine Learning)

Model used:

- Isolation Forest
- Contamination = 0.05 (5% anomalies expected)

Features used:

- compute_cost
- storage_cost
- transfer_cost
- api_cost
- total_cost

Model execution:

```
model = IsolationForest(contamination=0.05, random_state=42)
df['anomaly'] = model.fit_predict(features)
```

Output:

- 1 → Normal
- -1 → Anomaly

Mapped to:

Normal / Anomaly labels

Step 7: Recommendation Engine

Row-wise logic implemented using function:

Priority order:

1. If anomaly → "Investigate abnormal cost spike"
2. If waste → "Optimize or remove underutilized resources"
3. If compute_cost > 50% → "Use reserved instances"
4. If storage_cost > 40% → "Optimize storage usage"
5. If api_cost > 30% → "Reduce API calls"
6. Else → "Monitor usage"

Step 8: KPI Calculation

Metrics displayed:

- Total Cost → sum(total_cost)
- Waste Cost → sum(waste_cost)
- Anomaly Count → count(anomaly == -1)
- Waste Resources → count(is_waste == True)

Step 9: Visualization Logic

- Bar Chart:
    - Cost by department
    - Cost breakdown
- Scatter Plot:
    - X-axis → index

    - Y-axis → total_cost
    - Color separation → anomaly vs normal
- Tables:
    - Top waste resources
    - Recommendations

2.3 Information Flow

The system follows a transformation pipeline:

Raw CSV
↓
Feature Engineering
↓
Filtered Data
↓
Waste Detection
↓
Anomaly Detection
↓
Recommendation Engine
↓
Dashboard Output

2.4 Components Design

1. Data Loader

- Reads CSV file
- Validates structure

2. Feature Engineering Module

- Adds cost columns
- Computes total cost
- Maps service types

3. Waste Detection Module

- Computes quantiles
- Applies rule-based conditions

- Generates binary waste flag

4. Anomaly Detection Module

- Trains Isolation Forest
- Predicts anomalies
- Labels data

5. Recommendation Engine

- Applies conditional logic
- Generates action suggestions

6. Visualization Module

- Uses Streamlit + Matplotlib
- Displays KPIs, charts, tables

2.5 Key Design Considerations

- Choice of quantile thresholds for flexibility
- Fixed cost rates for simplicity
- Lightweight ML model for performance
- Sequential pipeline for clarity

2.6 API Catalogue

No APIs used. Entire logic is implemented internally using Python.

3. Data Design

3.1 Data Model

Input Columns:

- compute_hours
- storage_usage_gb
- data_transfer_gb
- api_requests
- cpu_utilization

- active_users

Derived Columns:

- compute_cost
- storage_cost
- transfer_cost
- api_cost
- total_cost

Flags:

- is_waste
- anomaly

Output:

- recommendation

3.2 Data Access Mechanism

- Single CSV read
- In-memory processing
- No database

3.3 Data Retention Policies

- No persistence
- Temporary session-based data

3.4 Data Migration

- Not required

4. Interfaces

- Sidebar → filters
- Main dashboard → charts + KPIs
- Tables → detailed insights

5. State and Session Management

- Streamlit reruns script on interaction
- State maintained implicitly

6. Caching

- Not implemented
- Can be added using:

```
@st.cache_data
```

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data
- Local execution

7.2 Performance Aspects

- Linear time complexity
- Efficient Pandas operations
- Fast ML inference

8. References

- Scikit-learn (Isolation Forest)
- Pandas
- Streamlit